



CCODEL
RAISING A NEW GENERATION OF
GLOBAL LEADERS

Computer Programming I (Java 1)

COS221

Mr. Otavie Okuoyo

Phd (in view Computer Science)

College of Science and Technology



Visit Our Website

ccodel.covenantuniversity.edu.ng



MODULE FIVE: METHODS

UNIT ONE: METHODS AND PARAMETER PASSING

METHODS AND PARAMETER PASSING

Learning Objectives

By the end of this unit, you will be able to:

- **identify the key parts of a method.**
- **define void and non-void methods, and demonstrate how to call them.**
- **compare formal parameters with actual arguments using simple examples.**
- **differentiate between static and instance methods through code analysis.**
- **implement method overloading by writing multiple methods with the same name but different parameters.**

Learning Outcomes

At the end of this unit, you should be able to:

- **understand the purpose and structure of methods.**
- **define and invoke both void and non-void methods.**
- **differentiate between formal parameters and actual arguments.**
- **differentiate between static and instance methods.**
- **apply method overloading for flexible code.**

Introduction

- In this unit, we dive into one of the core building blocks of Java programming, methods. You will learn how methods are defined, how to pass data into them using parameters, and how to return results.
- By exploring user-defined methods, you will begin to see how modular code can make your programs more organized, reusable, and easier to manage. We will also break down key concepts such as method signatures, variable scope, and how methods are invoked.

What is a Method?

A method is a reusable block of code that performs a specific task. Methods helps in:

- 1. Modularizing code: Breaking code into smaller, manageable parts.**
- 2. Improving readability: Logical separation of tasks.**
- 3. Reducing redundancy: Avoidance of repetition, write once and you use many times.**

The concept of write once and use many times can be demonstrated with the code block below.

What is a Method? (2)

```
public class ShapeDetails {  
    static void shapeInfo(int len, int width) {  
        int area = len * width;  
        int perimeter = 2 * (len + width);  
        System.out.println("Length of the shape: " + len);  
        System.out.println("Width of the shape: " + width);  
        System.out.println("Area: " + area);  
        System.out.println("Perimeter: " + perimeter);  
        System.out.println();  
    }  
  
    public static void main(String[] args) {  
        shapeInfo(5, 10);    // First use  
        shapeInfo(3, 7);    // Second use  
        shapeInfo(8, 4);    // Third use  
        shapeInfo(6, 6);    // Fourth use (Square)  
    }  
}
```

What is a Method? (2)

- In the code above, the *shapeInfo()* method demonstrates how methods *promote modularity* and *reduce repetition*.
- Instead of rewriting the logic for each shape, the method is reused with different length and width values to calculate and display area and perimeter.
- This approach makes the code more organized, easier to read, and simpler to maintain, since any changes to the calculation logic only need to be made in one place.

Void Methods vs Non Void Methods

- ***Void methods*** perform actions but do not return any value. They use ***void*** as the return type. The syntax is:

```
accessModifier void methodName (parameterList) {  
    // method body  
}
```

- ***Non void (returning) methods*** are methods that return a value. The syntax is:

```
accessModifier dataType methodName (parameterList) {  
    // method logic  
    return value;  
}
```

Void Method - Example 1

```
1  public class Main {  
2      public static void sayHello (String name) {  
3          System.out.print("Hello " + name);  
4      }  
5  
6      public static void main(String[] args) {  
7          sayHello("James");  
8      }  
9  }
```

The method does not return any value. In line 7, the method, *sayHello()* is called with “*James*” as argument. It will output:

Hello James

Void Method - Example 2

```
public class MathOperations {  
    public static void printSum(int a, int b) {  
        int sum = a + b;  
        System.out.println(  
            "The sum of " + a + " and " + b + " is " + sum);  
    }  
  
    public static void main(String[] args) {  
        printSum(5, 10);  
    }  
}
```

The ***MathOperations*** class has a void method, ***printSum()*** that adds two numbers and prints their sum. The ***main()*** calls it with **5** and **10** passed as parameters. The output is:

The sum of 5 and 10 is 15

Return Method - Example 1

```
public class SquareUtils {  
    public static int square(int number) {  
        return number * number;  
    }  
  
    public static void main(String[] args) {  
        System.out.println("Square: " + square(5));  
    }  
}
```

The class *SquareUtils* has a method called *square()* that returns the square of a number. The *main* method prints the square of 5. The output is:

Square: 25

Return Method - Example 2

```
public class Greeting {  
    public static String getGreeting(String name) {  
        return "Hello, " + name + "!";  
    }  
  
    public static void main(String[] args) {  
        String message = getGreeting("Oghenetega");  
        System.out.println(message);  
    }  
}
```

The class ***Greeting*** has a method ***getGreeting()*** that takes a name and returns a personalized greeting message. The main method calls ***getGreeting("Oghenetega")*** with the name "***Oghenetega***" and the output is: ***Hello, Oghenetega!***

Parameterized Methods vs Non-Parameterized Methods

- ***Parameterized methods*** are methods that take one or more parameters to perform operation(s). The syntax is:

```
accessModifier returnType methodName (parameterList) {  
    // method body  
}
```

- ***Non-parameterized methods*** are methods that do not require parameters to operate. The syntax is:

```
accessModifier returnType methodName () {  
    // method body  
}
```

Parameterized Method - Example 1

```
public class PersonInfo {  
    public static String getAgeInfo(int age, String name) {  
        return name + " is " + age + " years old.";  
    }  
  
    public static void main(String[] args) {  
        String result = getAgeInfo(45, "Blake");  
        System.out.println(result);  
    }  
}
```

The ***PersonInfo*** class has a method, ***getAgeInfo()*** that takes a name and age and returns a sentence with the name and the age. When ***getAgeInfo(45, "Blake")*** is called, it outputs:

Blake is 45 years old.

Parameterized Method - Example 2

```
public class FullName {  
    public static String getFullName(String firstName, String lastName) {  
        return firstName + " " + lastName;  
    }  
  
    public static void main(String[] args) {  
        System.out.println("Full Name: " + getFullName("John", "Doe"));  
    }  
}
```

The class ***FullName*** contains a method ***getFullName()*** that takes a first name and last name as parameters and returns the full name as a single string. When ***getFullName()*** is called with "***John***" and "***Doe***" it outputs:

Full Name: John Doe

Non-Parameterized Method - Example 1

```
import java.time.LocalDate;  
import java.time.LocalTime;  
public class DatePrinter {  
    public static void printDate() {  
        System.out.println("Today's date: " + LocalDate.now());  
        System.out.println("Current Time: " + LocalTime.now());  
    }  
    public static void main(String[] args) {  
        printDate();  
    }  
}
```

The code prints current date and time using the *LocalDate* and *LocalTime* classes from the *java.time* package. Output:

Today's date: 2025-06-09

Current Time: 01:26:39.633413700

Non-Parameterized Method - Example 2

```
import java.util.Random;
public class RandomGenerator {
    public static int generate() {
        Random r = new Random();
        return r.nextInt(100); // 0 to 99
    }

    public static void main(String[] args) {
        System.out.println("Random Number: " + generate());
    }
}
```

The code defines a method ***generate()*** that generates and prints a random number between **0** and **99**. The output will be in the format shown below:

Random Number: <Random Number>

Static vs Non-Static Methods

- ***Static methods*** are methods that belong to the class, not to instances (objects). They can be called without creating an object of the class. They can only directly access other static methods or variables. Commonly used for utility or helper methods.
- ***Non-static methods*** are methods that belong to an object (instance) of the class. They require you to create an object to call them. They can access both static and non-static members. They are used when behavior depends on object state.

Static Methods

- All the methods we have been using from void to return methods and parameterized to non-parameterized methods are all examples of static methods
- In each method we have used, just before the keyword the return type, you will see the keyword **static**, which indicates that the method is static.

Non-Static Methods - Example 1

```
class Person {  
    String name;  
  
    public void greet() {  
        System.out.println("Hello, " + name + "!");  
    }  
  
    public static void main(String[] args) {  
        Person p = new Person();  
        p.name = "Alice";  
        p.greet();  
    }  
}
```

This code defines a **Person** class with a name and a non-static method, ***greet()*** that prints a greeting.

In the ***main()*** method, it creates a **Person**, sets the name to "***Alice***", and prints: ***Hello, Alice!***

Non-Static Methods - Example 2

```
class Rectangle {  
    int length, width;  
  
    public void setDimensions(int l, int w) {  
        length = l;  
        width = w;  
    }  
  
    public int getArea() {  
        return length * width;  
    }  
  
    public static void main(String[] args) {  
        Rectangle r = new Rectangle();  
        r.setDimensions(5, 3);  
        System.out.println("Area: " + r.getArea());  
    }  
}
```

- This code defines the ***setDimensions()*** method that sets the dimensions, and ***getArea()*** returns the area.
- In ***main()***, a rectangle object is created, dimensions are set to **5** and **3**, and the area (**15**) is printed.

Non-Static Methods - Example 3

```
class Employee {  
    double baseSalary, bonus;  
  
    public double totalPay() {  
        return baseSalary + bonus;  
    }  
  
    public void applyRaise(double percentage) {  
        baseSalary += baseSalary * (percentage / 100);  
    }  
  
    public static void main(String[] args) {  
        Employee e = new Employee();  
        e.baseSalary = 50000;  
        e.bonus = 5000;  
        e.applyRaise(10); // Raise by 10%  
        System.out.println("Total Salary: " + e.totalPay());  
    }  
}
```

- This code defines the ***totalPay()*** and ***applyRaise()*** methods.
- In ***main()***, the salary of an employee is set, raised by 10%, and the updated total salary (60000) is printed.

Parameters Passing in Java

Parameter passing refers to the mechanism of transferring data between methods. The two types of parameter passing are:

- 1. Pass-by-value (primitive types):*** A copy of the value is passed. Changes inside the method do not affect the original variable.
- 2. Pass-by-reference (objects/arrays):*** The memory address (reference) is passed. Changes inside the method affect the original object.

Pass-by-Value - Example 1

```
public class PassByValue {  
    static void modifyValue(int number) {  
        number = number + 10;  
        System.out.println("Inside method: " + number);  
    }  
  
    public static void main(String[] args) {  
        int myNumber = 15;  
        System.out.println("Before method call: " + myNumber);  
        modifyValue(myNumber);  
        System.out.println("After method call: " + myNumber);  
    }  
}
```

Pass-by-Value - Example 1 (2)

- The output of the above code is as follows:

Before method call: 15

Inside method: 25

After method call: 15

- We started with *myNumber* set to *15* and passed it to *modifyValue*. Since Java uses pass-by-value, only a copy of the value (*15*) is changed (to *25*), while the original *myNumber* stays *15*. The code below also shows another example of pass-by-value.

Pass-by-Value - Example 2

```
1  public class PassByValueChar {
2      static void modifyChar(char x) {
3          x = 'Z';
4          System.out.println("Inside method: " + x);
5      }
6
7      public static void main(String[] args) {
8          char y = 'A';
9          System.out.println("Before method call: " + y);
10         modifyChar(y);
11         System.out.println("After method call: " + y);
12     }
13 }
```

Pass-by-Value - Example 2 (2)

- The output of the above code is:

Before method call: A

Inside method: Z

After method call: A

- The ***modifyChar(char c)*** method receives a copy of the char value, changes it to 'Z', and prints it. However, the original variable ***y*** in the ***main()*** method remains unchanged because Java passes primitives by value.

Pass-by-Reference - Example 1

```
1  public class PassByReferenceArray {
2      static void modifyArray(int[] arr) {
3          arr[0] = 99;
4          System.out.println("Inside Method: " + arr[0]);
5      }
6
7      public static void main(String[] args) {
8          int[] numbers = {1, 2, 3};
9          System.out.println("Before Method Call: " + numbers[0]);
10         modifyArray(numbers);
11         System.out.println("After Method Call: " + numbers[0]);
12     }
13 }
```

Pass-by-Reference - Example 1 (2)

- In the code above, the array *numbers* is passed to the *modifyArray()* method, where its first element is set to **99**. Because arrays are passed by reference, this change affects the original array in the *main()* method. However, reassigning the array inside the method will not affect the original.
- The output of the code above is as follows:

Before Method Call: 1

Inside Method: 99

After Method Call: 99

Pass-by-Reference - Example 2

```
public class PassByReference {  
    static void changeFirstVal(int[] num) {  
        num[0] = 100;  
        System.out.println("Inside Method: " + num[0]);  
    }  
    public static void main(String[] args) {  
        int[] numbers = {2, 4, 6};  
        System.out.println("Before method call: " + numbers[0]);  
        changeFirstVal(numbers);  
        System.out.println("After method call: " + numbers[0]);  
    }  
}
```

Pass-by-Reference - Example 2 (2)

- This code shows that when an array is passed to a method, changes to its elements inside the method affect the original array. The method *changeFirstVal()* sets the first element to **100**, and this change is visible outside the method because arrays are passed by reference in Java.
- The output of the code above is as follows:

Before method call: 2

Inside Method: 100

After method call: 100

Method Overloading

- **This is a feature that allows a class to have more than one method with the same name but with different parameter lists. The Java compiler selects the appropriate method based on:**
 - 1. Method signature (name + parameter list).**
 - 2. Arguments passed during the method call.**

Method Overloading - Example 1

```
public class PrintDisplay {  
    static void print (String name) {  
        System.out.println("I am " + name);  
    }  
  
    static void print (int age) {  
        System.out.println("I am " + age + " years old");  
    }  
  
    public static void main(String[] args) {  
        print ("John"); // Calls the first method  
        print (18);     // Calls the second method  
    }  
}
```

Method Overloading - Example 1 (2)

- The code demonstrates method overloading in using two print methods with different parameter types (String and int). Both are static, allowing direct calls from main.

print("John") outputs: *I am John*

print(18) outputs: *I am 18 years old*

- This shows how Java allows method overloading: methods with the same name but different parameter types.

Method Overloading - Example 2

```
class PlusCalculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
    double add(double a, double b) {  
        return a + b;  
    }  
}  
  
public class Calculator {  
    public static void main(String[] args) {  
        PlusCalculator calc = new PlusCalculator();  
        System.out.println(calc.add(5, 10));  
        System.out.println(calc.add(2, 3, 4));  
        System.out.println(calc.add(1.5, 2.5));  
    }  
}
```

Method Overloading - Example 2 (2)

- The ***Plus*** class defines three ***add()*** methods with different parameter lists (two ints, three ints, and two doubles).
- In ***main()***, the correct method is chosen based on the arguments passed.
- The output of the code is:

15

9

4.0

Type Promotion

- In Java, when an exact data type match is not found during operations or method calls, smaller data types are automatically promoted to larger ones to ensure compatibility. The promotion order is:

byte => short => int => long => float => double

char => int

Type Promotion - Example 1

```
public class TypePromotion {  
    static void display (int x) {  
        System.out.println("Int: " + x);  
    }  
  
    public static void main(String[] args) {  
        display('A');  
    }  
}
```

- In the code above, the output is **65** because the character '**A**' is automatically promoted to its Unicode integer value. In Java, characters are internally represented using Unicode, and '**A**' corresponds to **65**.

Type Promotion - Example 2

```
class ExampleFive {  
    void test(double x) {  
        System.out.println("double method: " + x);  
    }  
  
    public static void main(String[] args) {  
        short s = 2;  
        new ExampleFive().test(s);  
    }  
}
```

- In the code above, **short** is promoted through multiple steps. Short is promoted through int, then through long, through float and finally to double. Since only double is defined, Java promotes step-by-step until it fits.

In-Text Questions (ITQs)

- 1. What is the main purpose of using methods in Java?**
- 2. What happens when you pass a primitive type to a method?**
- 3. Can Java promote data types automatically in method overloading?**
- 4. What is the significance of static methods?**

In-Text Answers (ITAs)

- 1. Methods in Java are used to modularize code, improve readability, and eliminate redundancy by allowing us to write code once and reuse it multiple times.**
- 2. When a primitive type is passed to a method, only a copy of the value is used. Any changes inside the method do not affect the original variable.**
- 3. Yes, Java can promote smaller data types to larger ones (like byte to int) if no exact match is found for the method parameters during overloading.**
- 4. Static methods belong to the class rather than an instance and can be called without creating an object of the class.**

Conclusion

- **Understanding methods and parameter passing is key to writing clean, modular, and efficient Java programs. Methods allow code reuse, separation of concerns, and better complexity management.**
- **Grasping void vs. return-type methods, static vs. non-static methods, and parameter passing builds a strong base for advanced topics like object-oriented programming. Method overloading further improves flexibility by supporting multiple versions of a method for different inputs.**

Summary

- **This unit introduced methods, showing how they organize code into reusable blocks for specific tasks. You learned about void and non-void methods, parameterized and non-parameterized methods, and the difference between static and non-static methods.**
- **The unit covered parameter passing: Pass-by-value for primitives and pass-by-reference.**
- **Finally, we explored method overloading and how Java uses type promotion to match method calls.**